

遥感图像地物分类流程

1. 制作标签

使用arcgis pro或者arcgis或者envi，画标签，保存为tiff格式

2. 处理标签数据

用python gdal库安装 osgdal库，如果安装失败就需要下载 对应库得 .whl去安装，网站具体搞忘了，可以百度

或者rasterio库

2.1 读入tif数据

```
def readTif(fileName):  
    """  
    dataset包含了tif文件得属性比如  
    波段数  
    高  
    宽  
    数据  
    """  
    dataset = rasterio.open(fileName)  
    if dataset == None:  
        print(fileName + "文件无法打开")  
        return None  
    # print(dataset.width)  
  
    return dataset
```

2.2 处理数据

```
import csv  
# 提取栅格图像信息,制作数据  
ori_dataset = readTif(organ_path)  
label_dataset = readTif(sample_path)  
  
width = ori_dataset.width # 宽  
height = ori_dataset.height # 高  
  
bands = ori_dataset.count # 波段数  
# ori_data = for k in range(bands)  
  
label_matri = label_dataset.read(1) #读出标签的矩阵  
data_matri = ori_dataset.read() #原始图像的矩阵  
  
count = np.count_nonzero(label_matri) #非零就是标签，有多少非零的就代表样本像素是  
多少
```

```

print(count)
train_data = np.zeros((count, 8), dtype=data_matri.dtype) # 新建一个count*8的
numpy数组，第8维度是原始图像的某一像素点对应的标签，0~6代表这一个像素点对应的7ge波
段，landsata影像
nonzero_indices = np.nonzero(label_matri) #非零索引，返回的是
"""
(row:array([ 30,  31,  31, ..., 390, 390, 390], dtype=int64), col:array([166,
165, 166, ..., 186, 187, 188], dtype=int64))
"""
print(nonzero_indices)
# 写入数据csv，提取训练数据
# 将 train_data 写入 CSV 文件
csv_file = open(csv_filename, mode='w', newline='')
csv_writer = csv.writer(csv_file)
# 写入 CSV 文件的标题行，包括 Label 和 LabelName
csv_writer.writerow(csv_head_name)

for i in range(count):
    print(i)
    row, col = nonzero_indices[0][i], nonzero_indices[1][i]
    train_data[i, :7] = data_matri[:, row, col]
    train_data[i, 7] = label_matri[row, col]
    label = int(train_data[i, 7])
    row_data = train_data[i]
    row_data = np.append(row_data, labels_name[label]) # 在数据行中添加
LabelName
    csv_writer.writerow(row_data)

print(f"已将数据写入 CSV 文件: {csv_filename}")
csv_file.close()

```

2.3 数据格式

生成的数据格式如下

```

Band1,Band2,Band3,Band4,Band5,Band6,Band7,Label,LabelName
812,774,969,1111,1152,1146,1069,2,building
801,755,846,1016,1177,1411,1472,2,building
794,748,949,1179,1202,1399,1383,2,building
605,567,691,877,1537,1880,2070,2,building
602,556,768,994,1506,1625,1607,2,building
613,570,768,1045,1394,1483,1460,2,building
465,408,562,772,963,1035,990,2,building
549,484,648,828,969,1096,1028,2,building

```

3. 训练

```

from sklearn.ensemble import RandomForestClassifier
from sklearn import model_selection
import pickle

```

```

X = train_data[:, :7]
Y = train_data[:, 7]
# print(X.shape)
# print(Y.shape)
X_train, X_test, y_train, y_test = model_selection.train_test_split(X, Y,
test_size=0.1, random_state=42, stratify=Y)
print(y_train)
# 3.用100个树来创建随机森林模型·训练随机森林
classifier = RandomForestClassifier(n_estimators=100,
                                   bootstrap = True,
                                   max_features = 'sqrt')
classifier.fit(X_train, y_train)

# 4.计算随机森林的准确率
print("训练集:", classifier.score(X_train, y_train))
print("测试集:", classifier.score(X_test, y_test))

pred_test_y = classifier.predict(X_test)
cfm = CFM(5, labels_name)
cfm.update(pred_test_y, y_test)
acc, comment_numpy = cfm.get_cfm()
print(comment_numpy)
cfm.plot()

file = open(model_path, "wb")
#将模型写入文件:
pickle.dump(classifier, file)
#最后关闭文件:
file.close()

```

4. 使用模型预测

```

pred_dataset = readTif(pred_path)
pred_width = pred_dataset.width
pred_height = pred_dataset.height
pred_bands = pred_dataset.count
pred_geotrans = pred_dataset.transform
pred_crs = pred_dataset.crs

print(pred_geotrans)
print(pred_crs)

file = open(model_path, "rb")
# 把模型从文件中读取出来
rf_model = pickle.load(file)
# 关闭文件
file.close()

pred_martix = pred_dataset.read()
data = np.zeros((pred_martix.shape[0], pred_martix.shape[1] *
pred_martix.shape[2]))

```

```

# print(pred_martix.shape)
# print(pred_martix[0])
for i in range(pred_martix.shape[0]):
    # 第i个波段一维数组
    data[i] = pred_martix[i].flatten()
# 转换下维度
pred_x = data.swapaxes(0, 1)

pred_y = rf_model.predict(pred_x)
# print(pred_y, pred_y.shape)

# 将标签还原为图像的二维矩阵
pred_image = pred_y.reshape(pred_martix.shape[1], pred_martix.shape[2])
height_, width_ = pred_image.shape
tif_data = np.zeros((height_, width_, 3), dtype=np.int64)
for label, color in color_mapping.items():
    tif_data[pred_image == label] = color

tif_data = np.transpose(tif_data, (2, 0, 1))

im_bands, im_height, im_width = tif_data.shape
driver = gdal.GetDriverByName("GTiff")
dataset = driver.Create(pred_result_tif_path, im_width, im_height, im_bands,
gdal.GDT_Byte)
for i in range(im_bands):
    dataset.GetRasterBand(i + 1).WriteArray(tif_data[i])
# if dataset != None:
#     #将栅格数据和地理坐标系统关联起来
#     dataset.SetProjection(pred_crs) # 写入投影
#     dataset.SetGeoTransform(pred_geotrans) # 写入仿射变换参数

dataset = None

```

5. other

```

import numpy as np
import matplotlib.pyplot as plt
from prettytable import PrettyTable

class CFM:
    """
    混淆矩阵类
    返回精度和混淆举证
    """
    def __init__(self, num_classes: int, labels: list):
        self.matrix = np.zeros((num_classes, num_classes))
        self.num_classes = num_classes
        self.labels = labels

    def plot(self):
        matrix = self.matrix
        print(matrix)
        plt.imshow(matrix, cmap=plt.cm.Blues)

```

```

# 设置x轴坐标label
plt.xticks(range(self.num_classes), self.labels, rotation=45)
# 设置y轴坐标label
plt.yticks(range(self.num_classes), self.labels)
# 显示colorbar
plt.colorbar()
plt.xlabel('True Labels')
plt.ylabel('Predicted Labels')
plt.title('Confusion matrix')

# 在图中标注数量/概率信息
thresh = matrix.max() / 2
for x in range(self.num_classes):
    for y in range(self.num_classes):
        # 注意这里的matrix[y, x]不是matrix[x, y]
        info = int(matrix[y, x])
        plt.text(x, y, info,
                  verticalalignment='center',
                  horizontalalignment='center',
                  color="white" if info > thresh else "black")
plt.tight_layout()
plt.show()

def update(self, preds, labels):
    """_summary_

    Args:
        preds (_type_): _description_
        labels (_type_): _description_

    preds:预测值
    labels:真实值
    confusion martix
        label0 label1 label2 label3
    pred0
    pred1
    pred2
    pred3
    """
    for p, t in zip(preds, labels):
        self.matrix[p, t] += 1
    print("confusion matrix", self.matrix)

def get_cfm(self):
    """
    Accuracy: 正确样本占总样本数量的比例
    Percision: 精度Precision
    Recall: 召回率
    Specificality: 特异性
    """
    sum_true = 0
    for i in range(self.num_classes):
        sum_true += self.matrix[i, i]
    acc = sum_true / np.sum(self.matrix)
    print("the model accuracy is ", acc)

```

```

comment_labels = ["category", "Precision", "Recall", "Specificity"]
tabel = PrettyTable()
tabel.field_names = comment_labels
comment_numpy = np.zeros((self.num_classes, 3))
for i in range(self.num_classes):
    # 第i个分类的精确率，召回率，特异度
    TP = self.matrix[i, i]
    FP = np.sum(self.matrix[i, :]) - TP
    FN = np.sum(self.matrix[:, i]) - TP
    TN = np.sum(self.matrix) - TP - FN - FP
    # 保留三位小数，如果 TP + FN 不等于零，就计算并将结果四舍五入到小数点
    # 后三位；否则，率设置为0。
    Precision = round(TP / (TP + FP), 3) if TP + FP != 0 else 0.
    Recall = round(TP / (TP + FN), 3) if TP + FN != 0 else 0.
    Specificity = round(TN / (TN + FP), 3) if TN + FP != 0 else 0.
    tabel.add_row([self.labels[i], Precision, Recall, Specificity])
    comment_numpy[i] = [Precision, Recall, Specificity]
print(tabel)
return acc, comment_numpy

if __name__ == "__main__":
    cfm = CFM(2, ["cat", "dog"])
    actual = [1, 0, 1, 1, 0, 1, 0, 0, 1, 0]
    predicted = [1, 0, 1, 0, 0, 1, 1, 1, 1, 0]
    cfm.update(predicted, actual)
    acc, comment_numpy = cfm.get_cfm()
    print(comment_numpy)
    cfm.plot()

```

```

sample_path = "../sample/sample.tif" #标签图
origin_path = "../datasets/landsat.tif" #原始图
pred_path = "../datasets/landsat.tif" #需要预测的图
txt_Path = "../result/label_data.txt" #无
labels_name = ["", "tudi", "building", "veg", "water"] # 样本名字，分类的类别
csv_filename = '../result/train_data.csv' # 生成训练数据的存放路径
csv_head_name = ['Band1', 'Band2', 'Band3', 'Band4', 'Band5', 'Band6',
'Band7', 'Label', 'LabelName'] # 存放格式
model_path = "../model/myrnrf.pickle" # 最终保存的模型路径
pred_result_tif_path = "../result/pred_landsat.tif" # 用训练的模型保存的路径
color_mapping = {
    1: (255, 255, 0),
    2: (255, 0, 0),
    3: (0, 255, 0),
    4: (0, 0, 255)
}
# 颜色映射从2D标签映射到3D

```